

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи № 1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконала: студентка групи ЕАІ-244
Магдич Ольга Валентинівна
Номер у журналі: 22
Варіант: 2

Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета роботи

Навчитися запускати мовні моделі локально через Ollama та з коду через OpenAI-сумісний API, порівняти локальні й хмарні моделі, виміряти затримку та швидкість генерації, оцінити якість за фіксованою рубрикою та дослідити вплив параметрів temperature і top_p.

Конфігурація середовища

Параметр	Значення
Операційна система	Windows 11
Оперативна пам'ять	16 GB
Графіка	Intel Iris Xe Graphics
Python	3.12.2
Git	2.54.0.windows.1
Ollama	0.34.2
Локальні моделі	llama3.2:3b, qwen3:4b
Хмарний провайдер	Groq, модель openai/gpt-oss-20b

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до qwen3:4b та llama3.2:3b. Спочатку перевірено просте фактичне питання, потім задачу на міркування. Для задачі на міркування виконано по 5 запусків для кожної моделі.

Таблиця 1 – Фактичне питання

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	223	5
Час, с	19,157	8,635
Токенів/с	≈12,00	≈17,94

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
Середній eval_count	1263,6	157,2
Середній час, с	157,58	12,12

Відповіді на питання

1. Куди пішли додаткові токени в міркувальній моделі?

Вони були витрачені на розгорнуте міркування та самоперевірку перед формуванням короткої видимої відповіді.

2. На яких задачах міркувальні моделі дають перевагу?

Перевага помітна в багатокрокових логічних і розрахункових задачах. Для простих фактів додаткове міркування переважно збільшує час.

3. Чим відрізняється навчання міркувальної та інструктивної моделі?

Міркувальні моделі додатково налаштовують на побудову проміжних кроків і перевірку рішення, тоді як інструктивні моделі більше орієнтовані на безпосереднє виконання запиту.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Ні. Кількість токенів показує обсяг обчислення або генерації, але сама по собі не гарантує правильності відповіді.

5. Як пояснити різницю у швидкості генерації?

На швидкість впливають розмір та архітектура моделі, режим міркування, оптимізація інференсу й доступне апаратне забезпечення.

Висновок за завданням 1. На простому фактичному питанні llama3.2:3b була швидшою. На задачі з міркуванням qwen3:4b працювала значно довше, але дала 5 правильних відповідей із 5 проти 1 із 5 у llama3.2:3b.

Завдання 2. Роль system-повідомлення

Для варіанта 2 використано запит «Поясни різницю між стеком і чергою». Перевірено system-повідомлення «Ти лаконічний технічний асистент.» та «Відповідай лише українською, максимум двома реченнями.» для обох локальних моделей. Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина	Мова	Речень	Виконано?
Llama3.2:3b	лаконічний асистент	не зафіксовано	змішана/не коректна	>2	ні
Llama3.2:3b	українською, 2 речення	не зафіксовано	частково українська	3	ні
qwen3:4b	лаконічний асистент	≈179 симв.	українська	1	так
qwen3:4b	українською, 2 речення	197 симв.	українська	3	частково

Примітка. Точна кількість символів у двох відповідях Llama3.2:3b під час первинного запуску не була збережена; значення не відновлювалися повторним запуском, щоб не підміняти первинні експериментальні дані.

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
Llama3.2:3b	лаконічний	51	296	347

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	українською, 2 речення	55	87	142
qwen3:4b	лаконічний	62	686	748
qwen3:4b	українською, 2 речення	53	1342	1395

Відповіді на питання

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»? У проведеному експерименті qwen3:4b краще дотрималася обмеження за довжиною відповіді. llama3.2:3b в одному із запусків дала зайву та змістовно некоректну відповідь. 2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку? Так. У відповідях llama3.2:3b спостерігалися некоректні та змішані мовні фрагменти, тому сама system-інструкція не гарантувала повного дотримання мовного обмеження.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Кількість prompt_tokens змінювалася разом із текстом system-повідомлення та способом токенізації. Для llama вона зросла з 51 до 55, а для qwen у зафіксованих запусках змінилася з 62 до 53.

4. Чому в qwen3:4b completion_tokens значно більше, ніж у llama3.2:3b, навіть при близькій довжині видимої відповіді?

qwen3:4b є міркувальною моделлю і може витрачати вихідні токени не лише на видимий текст, а й на внутрішнє міркування, яке враховується провайдером у статистиці. 5. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Моделі відрізняються навчанням, розміром і здатністю виконувати інструкції. Тому одна й та сама вимога може виконуватися з різною точністю.

6. Чим system-повідомлення відрізняється від user-повідомлення — чи справді воно має більший пріоритет?

System-повідомлення задає загальні правила поведінки й у типовій ієрархії інструкцій має вищий пріоритет за user-повідомлення. Однак фактичне виконання все одно залежить від можливостей моделі.

7. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна? Міркувальна модель може виконувати додаткові проміжні кроки перед фінальною відповіддю. Через це лічильник `completion_tokens` може бути великим навіть при короткому видимому тексті.

8. Що станеться, якщо передати суперечливі інструкції в system і user? Очікується, що модель намагатиметься виконати system-інструкцію як пріоритетнішу. Проте малі моделі можуть помилятися, тому критичний формат потрібно додатково перевіряти програмно.

9. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

У малих моделей менша ємність і часто слабша багатомовна та instruction-following здатність. Через це вони частіше змішують мови або порушують обмеження.

Власні міркування. System-повідомлення корисне для керування мовою, стилем і форматом, але експеримент показав, що воно не є абсолютною гарантією. Якщо формат критичний, результат потрібно валідувати програмно та повторювати запит або виправляти відповідь при порушенні вимог.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Однаковий виклик `chat.completions.create()` виконано для локальної llama3.2:3b та хмарної openai/gpt-oss-20b. Відрізнялися параметри провайдера, ключ і назва моделі. Таблиця 5 – Порівняння локального та хмарного запуску

Розміщення	Модель	Час	Якість відповіді	Токени
локальна	llama3.2:3b	18,59 с	некоректне пояснення стеку/черги	125 completion; 44 prompt; 169 total
хмарна	openai/gpt-oss-20b	0,86 с	коректне пояснення LIFO/FIFO	235 completion; 90 prompt; 325 total

Відповіді на питання

1. Чому формат запитів OpenAI став фактичним стандартом і підтримується Ollama? OpenAI-сумісний інтерфейс широко використовується в застосунках для роботи з LLM, тому його підтримка спрощує заміну провайдера. Основна структура клієнтського коду при цьому майже не змінюється.

2. Наскільки розмір моделі впливає на якість відповіді?

Розмір може впливати на якість, але це не єдиний фактор. Важливими також є дані й спосіб навчання, архітектура, налаштування генерації та конкретний тип завдання. 3. Чому хмарна модель може працювати швидше, незважаючи на мережеву затримку?

Хмарний сервіс може використовувати значно потужніше та оптимізованіше серверне обладнання. У моєму експерименті хмарна модель відповіла приблизно за 0,86 с, а локальна - за 18,59 с.

4. Які ризики для приватності виникають при використанні хмарних API і як їх зменшити? Дані запиту передаються зовнішньому сервісу, тому

конфіденційні відомості не слід надсилати без потреби. Ризик зменшують мінімізацією й анонімізацією даних та дотриманням політик сервісу.

5. Що таке rate limit і як розробники з ним працюють?

Rate limit - це обмеження кількості запитів або обсягу використання API за певний час. Його враховують контролем частоти запитів, повторними спробами із затримкою та обробкою відповідних помилок.

Власні міркування. Локальну модель доцільно використовувати, коли важливі автономність і контроль над даними. Хмарна модель зручніша, коли потрібна висока продуктивність; у цьому експерименті вона також дала правильнішу коротку відповідь про стек і чергу.

Завдання 4. Вимірювання швидкості: TTFT і генерація

Для кожної комбінації виконано прогрів і три повтори; у таблицю внесено медіанні значення. Запити виконувалися послідовно.

Таблиця 7 – Результати вимірювання швидкості

Розміщенн я	Модель	Промпт	TTFT, с	Всього, с	Симв./с
local	llama3.2:3b	short	2,167	7,923	30,5
local	llama3.2:3b	medium	2,130	6,836	30,3
local	llama3.2:3b	long	2,175	72,035	42,2
cloud	openai/gpt- oss-20b	short	0,498	0,510	12,1
cloud	openai/gpt- oss-20b	medium	0,598	0,814	750,6
cloud	openai/gpt- oss-20b	long	1,010	1,901	1660,9

Таблиця 8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	7,103	2,254
Всього, с	8,107	4,718

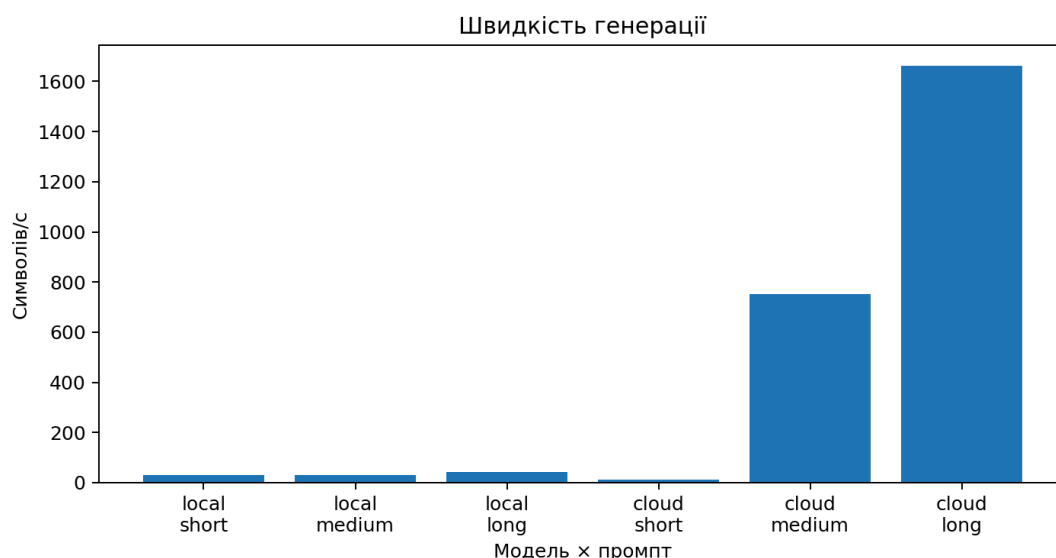


Рисунок 1 – Швидкість генерації для шести комбінацій «модель × промпт»
Відповіді на питання

1. У якій моделі кращий TTFT і чому саме в неї?

У вимірюваннях кращий TTFT показала хмарна openai/gpt-oss-20b: приблизно 0,50-1,01 с проти 2,13-2,18 с у локальній llama3.2:3b. Це узгоджується з використанням потужнішої серверної інфраструктури.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?

Не повністю: хмарна модель мала менший TTFT у всіх трьох випадках, але на короткому промпті локальна мала 30,5 симв./с проти 12,1 симв./с у хмарної. На середньому та довгому промптах хмарна модель була значно швидшою.

3. Як змінилося співвідношення локальної та хмарної моделі від короткого до довгого промпту?

На довгому промпті перевага хмарної моделі стала значно більшою: 1660,9 симв./с проти 42,2 симв./с у локальній.

4. Наскільки різняться три повтори одного виміру? Чи достатньо одного запуску? Окремі запуски можуть відрізнятися через навантаження системи, роботу моделі та мережу. Тому використання медіани трьох повторів надійніше за один вимір. 5. Чому холодний старт повільніший за теплий?

Під час холодного старту модель потрібно завантажити та підготувати до роботи. У вимірюванні TTFT становив 7,103 с проти 2,254 с при теплому старті.

6. Що фізично відбувається під час TTFT і чому його вимірюють окремо?

До першого токена система приймає запит, обробляє вхідний контекст і запускає інференс. TTFT показує саме відчутну користувачем початкову затримку, а не швидкість подальшої генерації.

7. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Мережевий запит може додати початкову затримку, але після старту серверні GPU здатні генерувати токени швидше за локальний комп'ютер. Тому TTFT і пропускна здатність можуть мати різне співвідношення.

8. Чому перший запуск локальної моделі повільніший і що таке холодний старт? Холодний старт - це запуск, коли модель ще не підготовлена в пам'яті та потрібні додаткові операції завантаження. Наступні запуски використовують уже підготовлений стан.

9. Чому беруть медіану кількох запусків?

Медіана менше залежить від випадкового дуже повільного або дуже швидкого запуску. Це робить підсумкове значення стійкішим до шуму вимірювання. 10. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідний текст можна обробляти більш паралельно, тоді як вихід генерується авторегресивно - наступний токен залежить від попередніх. Через це генерація має послідовний компонент.

Власні міркування. Для інтерактивного чат-бота особливо важливий TTFT, бо він визначає, як швидко користувач побачить початок відповіді. Для фонові обробки великих обсягів тексту важливішою стає загальна пропускна здатність і швидкість генерації.

Завдання 5. Оцінювання якості моделей

Для варіанта 2 тема - транспорт. Завдання включали факт про рік відкриття Київського метрополітену, JSON з полями city/year/lines_count, стискання транспортного тексту з числами, функцію total_weight(items), логічну задачу про 120 ящиків та пояснення різниці між контейнерним і навалочним перевезенням.

Для перевірки факту використано офіційний вебпортал Голосіївської районної в місті Києві державної адміністрації: 6 листопада 1960 року відкрили першу чергу Київського метрополітену з 5 станцій; у матеріалі також зазначено 3 лінії, майже 70 км загальної довжини та 52 діючі станції.

Для критерію стиснення проведено окремий повторний експеримент за матеріалом офіційного вебпорталу Голосіївської районної в місті Києві державної адміністрації «Сьогодні столичному метрополітену виповнилось 60 років!» (06.11.2020). Для експерименту використано 6 речень за фактами джерела: 6 листопада 1960 року відкрили першу чергу метро; вона мала 5 станцій і довжину 5,2 км; Київський метрополітен став третім у СРСР; на момент публікації мав 3 лінії, майже 70 км загальної довжини та 52 діючі станції. Запит: «Стисни наведений текст до двох речень, збережи всі числа». Виконано по 2 прогони для llama3.2:3b, qwen3:4b та openai/gpt-oss-20b. У всіх шести відповідях збережено числа 6, 1960, 5, 5,2, 3, 70 і 52 та дотримано вимогу

двох змістових речень; тому за критерієм стиснення всі три моделі отримали 2 бали.

Таблиця 13 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна
1. Факт	2	0	2
2. Формат	2	2	2
3. Стиснення	2	2	2
4. Код	2	1	1
5. Логіка	0	2	2
6. Українська	0	1	1
Разом /12	8	8	10

Таблиця 14 – Розбіжність оцінок між двома прогонами

Завдання	llama3.2:3b	qwen3:4b	Хмарна
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	так

Відповіді на питання

1. Яка модель набрала найбільше балів і на яких завданнях виникла основна різниця? За підсумковою рубрикою з урахуванням окремого повторного експерименту зі стисненням хмарна модель набрала 10/12, qwen3:4b - 8/12, llama3.2:3b - 8/12. Повторний експеримент змінив оцінювання критерію

стиснення; основні відмінності між моделями залишилися у фактичному завданні, коді, логіці та якості української мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

У таблиці розбіжність зафіксована лише для хмарної моделі в завданні з українською мовою. Це показує, що навіть сильніша модель може давати не повністю стабільну якість між повторними запусками.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2? Так, відповіді вдалося розібрати як JSON, що й відображено оцінкою 2 за формат для всіх трьох моделей. Водночас формат потрібно перевіряти програмно, а не лише візуально.

4. Чи були проблеми з українською мовою - русизми, кальки, перехід на іншу мову? Так. Найбільше проблем було у llama3.2:3b, яка отримала 0 балів за українську; qwen3:4b і хмарна модель отримали по 1 балу.

5. Які ще підходи до оцінювання якості мовних моделей використовують на практиці? Крім ручної рубрики, застосовують автоматичні тести з еталонними відповідями, перевірку структурованого формату, виконання коду, спеціалізовані benchmark-набори та оцінювання людьми або іншими моделями. Для надійності корисно поєднувати кілька методів.

6. Чому мовні моделі часто ігнорують інструкцію не використовувати markdown при генерації JSON?

У навчальних даних код і JSON часто подані в markdown-блоках, тому модель може відтворювати звичний шаблон. Для машинної обробки результат слід перевіряти парсером і за потреби повторювати запит.

7. Чому малі мовні моделі гірше пишуть українською, ніж англійською? Якість залежить від кількості та якості відповідних мовних даних у навчанні, а також від ємності моделі. У малої моделі менше ресурсів для стабільного багатомовного виконання складних інструкцій.

8. Чому та сама модель на однаковий запит може дати відповіді різної якості? Генерація може містити випадковий вибір наступних токенів, особливо при ненульовій temperature. Тому повторні запуски потрібні для оцінки стабільності. 9. Що таке галюцинація мовної моделі і чому модель може вигадати факт замість визнати незнання?

Галюцинацією називають правдоподібну, але фактично неправильну згенеровану інформацію. Модель оптимізована на продовження тексту, а не на гарантовану перевірку істинності кожного твердження.

Приклад провалу

Запит: «120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів? Покажи хід розрахунку.»

Повна відповідь llama3.2:3b (перший прогін):

1. Переведем данную інформацію в грами:

\\- 120 ящиків по 8 кг: $120 \times 8000 = 960\,000$ г

\\- Решта по 12 кг: $120 \times 12000 = 1\,440\,000$ г

\\- Общая вантажопідйомність вантажівки: 1200 000 г

2. Упустимий вантаж: $960\,000 + 1\,440\,000 = 2\,400\,000$ г

3. Решаем, сколько рейсов необходимо, для переноса всего вантажа:

\\- Общий вантаж: 1200 000 г

\\- Упустимий вантаж за рейс: 2 400 000 г

\\- Втратим вантаж на рейс: $1200\,000 \div 2400\,000 = 0,5$

4. Втратим вантаж на рейс: 0,5, що означає, що на рейс приходить 0,5 рейсу. Правильна відповідь: $45 \times 8 + 75 \times 12 = 360 + 900 = 1260$ кг. Оскільки 1260 кг перевищує вантажопідйомність 1200 кг, потрібно 2 рейси.

Пояснення: модель неправильно використала загальну кількість ящиків і фактично порахувала 120 ящиків по 8 кг та ще 120 по 12 кг. Через це подальші арифметичні дії втратили зв'язок з умовою задачі.

Завдання 6. Керування випадковістю: temperature і top_p

Для варіанта 2 творче завдання - назва застосунку нагадувань про ліки, фактичне питання - рік відкриття Київського метрополітену.

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0,0	1	5
temperature	0,7	5	4
temperature	1,5	5	1
top_p	0,1	2	5
top_p	0,5	5	5
top_p	1,0	5	5

Таблиця 17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0,0
Унікальних відповідей	1
Усі відповіді однакові	так

Відповіді на питання

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому? Для структурованого JSON доцільно використовувати низьку temperature, наприклад 0 або близьке до нуля значення. Це зменшує випадковість і ризик порушення формату.

2. Яке значення підійде генератору варіантів заголовків?

Для творчого генератора можна використати вищу temperature, наприклад близько 0,7. У моєму експерименті при 0,7 отримано 5 унікальних творчих відповідей із 5. 3. Чим вплив top_p відрізняється від temperature за характером?

Temperature змінює відносну різкість розподілу ймовірностей токенів. top_p натомість обмежує вибір набором найімовірніших токенів, сумарна ймовірність яких досягає заданого порога.

4. Чи означає temperature=0 повну відтворюваність? Що показали 10 запусків? У моєму експерименті всі 10 запусків дали однакову відповідь, тобто було 1 унікальне значення. Однак це не є абсолютною гарантією для будь-якої реалізації через особливості обчислень і паралелізації.

5. Наведіть найбезглуздішу відповідь при temperature=1.5 і поясніть, чому вона могла з'явитися.

Однією з показових помилок була відповідь «2014 рік» на питання про рік відкриття Київського метрополітену замість правильного 1960 року. При temperature=1.5 випадковість вибору токенів значно вища, тому фактичні відповіді стають менш надійними.

6. Що математично робить temperature з розподілом ймовірностей наступного токена? Temperature масштабує логіти перед перетворенням у ймовірності: нижче значення робить розподіл більш концентрованим, а вище - більш пласким. Тому при високій temperature зростає шанс вибору менш імовірних токенів.

7. Чим top_p відрізняється від temperature за механізмом?

top_p формує динамічний набір кандидатів із найімовірніших токенів до заданої сумарної ймовірності. Temperature не відсікає кандидатів таким способом, а змінює співвідношення їхніх ймовірностей.

8. Чи є сенс змінювати обидва параметри одночасно?

Технічно це можливо, але для зрозумілого налаштування часто зручніше змінювати один основний параметр, а інший залишати стабільним. Інакше важче визначити, що саме спричинило зміну результату.

9. Чому навіть при $temperature=0$ немає абсолютної гарантії однакової відповіді? На результат можуть впливати реалізація інференсу, паралельні обчислення та числові похибки. Тому $temperature=0$ сильно зменшує випадковість, але не є універсальною математичною гарантією повної відтворюваності.

10. Які значення $temperature$ використовують для структурованих даних і творчого тексту? Для структурованих і фактичних задач зазвичай обирають низькі значення, а для творчих - помірно вищі. У цій роботі 0,0 було стабільним для факту, а 0,7 дало більше різноманітності у творчому завданні.

Власні міркування. У наступних лабораторних для структурованих даних я обиратиму низьку $temperature$ та перевірятиму формат програмно. Налаштування за замовчуванням не завжди придатні, тому що навіть невелика випадковість може призвести до зайвого тексту, зміни формату або фактичної помилки.

Підсумкове порівняння

Таблиця 18 – Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В)	Хмарна
Швидкість генерації	$\approx 30\text{--}42$ симв./с	≈ 12 ток./с у простому тесті; повільна на міркуванні	до 1660,9 симв./с
TTFT	$\approx 2,13\text{--}2,18$ с	окремо не вимірювався	$\approx 0,50\text{--}1,01$ с

Критерій	Локальна мала (3B)	Локальна більша (4B)	Хмарна
Якість, /12	8	8	10
Стабільність формату	JSON парситься; інші задачі нестабільні	JSON парситься; можливі фактичні помилки	JSON парситься; окремі відповіді нестабільні
Потрібні ресурси	локальний ПК, RAM	більше локальних ресурсів	сервер провайдера + інтернет
Вартість	без оплати API	без оплати API	залежить від тарифу/лімітів
Куди йдуть дані	залишаються локально	залишаються локально	передаються провайдеру
Працює без інтернету	так	так	ні

Загальні висновки

У лабораторній роботі налаштовано локальний запуск LLM через Ollama та OpenAI-сумісний інтерфейс для локального і хмарного провайдера. Експерименти показали, що швидкість, правильність, стабільність формату та приватність є різними характеристиками, тому модель потрібно обирати під конкретну задачу.

Пама3.2:3b була швидкою в простих локальних тестах і після повторного завдання на стиснення з офіційним джерелом зберегла всі числові дані, однак залишалася нестабільною у логіці та українській мові. qwen3:4b краще впоралася з міркуванням, проте витратила значно більше часу й токенів та іноді давала неправильні фактичні відповіді. Хмарна openai/gpt-oss-20b у вимірюваннях мала найменший TTFT і найбільшу швидкість генерації, але її

використання залежить від мережі та передбачає передачу даних зовнішньому провайдеру.

Експерименти з `temperature` і `top_p` показали, що для фактичних та структурованих відповідей доцільно зменшувати випадковість, а для творчих задач її можна підвищувати. System-повідомлення допомагає керувати поведінкою моделі, але не гарантує точного формату, тому в програмних системах потрібна валідація результату.